



Project – Airline Reservation System

Object Oriented Programming

(CSEG1044)

Phase 3: Database and DAO layer

Members:-

Yash Jangir – 590014858

Parth Choudhary-590016913

Dushyant Singh-590012592

Soham Sharma-590017041

Anuj Pandey-590017258

1. Goal of the Phase

The primary goal of this phase is to design a relational database schema and implement a JDBC-based DAO (Data Access Object) layer for the Airline Reservation System. This phase ensures that:

- Data is stored in a structured format.
- Relationships between entities are properly maintained.
- Database operations are handled efficiently using Java.

2. Database Schema Design

The system consists of the following core tables adapted for the airline domain:

Routes

SQL

```
CREATE TABLE routes (  
    route_id INT PRIMARY KEY AUTO_INCREMENT,  
    origin VARCHAR(100),  
    destination VARCHAR(100)  
);
```

Passengers

SQL

```
CREATE TABLE passengers (  
    passenger_id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100),  
    email VARCHAR(100)  
);
```

Flights

SQL

```
CREATE TABLE flights (  
    flight_id INT PRIMARY KEY AUTO_INCREMENT,  
    route_id INT,  
    flight_date DATE,  
    FOREIGN KEY (route_id) REFERENCES routes(route_id)  
);
```

Bookings

SQL

```
CREATE TABLE bookings (  
    booking_id INT PRIMARY KEY AUTO_INCREMENT,  
    passenger_id INT,  
    flight_id INT,  
    status VARCHAR(50),
```

```
FOREIGN KEY (passenger_id) REFERENCES passengers(passenger_id),  
FOREIGN KEY (flight_id) REFERENCES flights(flight_id)  
);
```

Table Relationships:

- A Route can have multiple Flights (1-to-Many).
- A Passenger can make multiple Bookings (1-to-Many).
- A Flight can receive multiple Bookings (1-to-Many).

3. DAO Layer Design

The DAO layer is responsible for handling all database interactions.

DAO Classes Implemented:

- RouteDAO
- FlightDAO
- BookingDAO

Each DAO class:

- Uses JDBC for database interaction.
- Uses PreparedStatement for security.
- Contains CRUD operations.

4. CRUD Operations Implementation

RouteDAO

Handles Create, Read, Update, and Delete for routes.

Java

```
package dao;  
import java.sql.*;  
import db.DBConnection;  
  
public class RouteDAO {  
    // CREATE  
    public void addRoute(String origin, String destination) throws Exception {  
        Connection conn = DBConnection.getConnection();  
        String sql = "INSERT INTO routes (origin, destination) VALUES (?, ?)";  
        PreparedStatement ps = conn.prepareStatement(sql);  
        ps.setString(1, origin);  
        ps.setString(2, destination);  
        ps.executeUpdate();  
    }  
}
```

```

        conn.close();
    }

    // READ
    public void getAllRoutes() throws Exception {
        Connection conn = DBConnection.getConnection();
        String sql = "SELECT * FROM routes";
        PreparedStatement ps = conn.prepareStatement(sql);
        ResultSet rs = ps.executeQuery();
        while (rs.next()) {
            System.out.println(rs.getInt("route_id") + " | " +
                               rs.getString("origin") + " to " +
                               rs.getString("destination"));
        }
        conn.close();
    }

    // UPDATE & DELETE omitted for brevity, but follow the same PreparedStatement pattern.
}

```

FlightDAO

Handles Create and Read for flights.

Java

```

package dao;
import java.sql.*;
import db.DBConnection;

public class FlightDAO {
    // CREATE
    public void addFlight(int routeId, String flightDate) throws Exception {
        Connection conn = DBConnection.getConnection();
        String sql = "INSERT INTO flights (route_id, flight_date) VALUES (?, ?)";
        PreparedStatement ps = conn.prepareStatement(sql);
        ps.setInt(1, routeId);
        ps.setString(2, flightDate);
        ps.executeUpdate();
        conn.close();
    }

    // READ
    public void getAllFlights() throws Exception {
        Connection conn = DBConnection.getConnection();
        String sql = "SELECT * FROM flights";
        PreparedStatement ps = conn.prepareStatement(sql);
        ResultSet rs = ps.executeQuery();
        while (rs.next()) {
            System.out.println("Flight ID: " + rs.getInt("flight_id") +
                               " | Route ID: " + rs.getInt("route_id") +
                               " | Date: " + rs.getString("flight_date"));
        }
        conn.close();
    }
}

```

BookingDAO

Handles creating and viewing passenger bookings.

Java

```
package dao;
import java.sql.*;
import db.DBConnection;

public class BookingDAO {
    // CREATE (Book a Flight)
    public void bookFlight(int passengerId, int flightId, String status) throws Exception {
        Connection conn = DBConnection.getConnection();
        String sql = "INSERT INTO bookings (passenger_id, flight_id, status) VALUES (?, ?, ?)";
        PreparedStatement ps = conn.prepareStatement(sql);
        ps.setInt(1, passengerId);
        ps.setInt(2, flightId);
        ps.setString(3, status);
        ps.executeUpdate();
        conn.close();
    }

    // READ
    public void getBookings() throws Exception {
        Connection conn = DBConnection.getConnection();
        String sql = "SELECT * FROM bookings";
        PreparedStatement ps = conn.prepareStatement(sql);
        ResultSet rs = ps.executeQuery();
        while (rs.next()) {
            System.out.println("Booking ID: " + rs.getInt("booking_id") +
                               " | Pass ID: " + rs.getInt("passenger_id") +
                               " | Flight ID: " + rs.getInt("flight_id") +
                               " | Status: " + rs.getString("status"));
        }
        conn.close();
    }
}
```

5. JDBC Implementation

The project follows best JDBC practices:

- Centralized connection handling using a utility class.
- Usage of Prepared Statements.
- Proper ResultSet mapping.
- Separation of SQL from UI layer.

Java

```
package db;
import java.sql.*;
```

```

public class DBConnection {
    public static Connection getConnection() throws Exception {
        return DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/airline_reservation",
            "root",
            "1234"
        );
    }
}

```

6. Working Flow of the System

System Flow: User → Main Class (Input Controller/Logic Handler) → DAO Layer (RouteDAO, FlightDAO, BookingDAO) → DBConnection (JDBC) → MySQL Database.

Operational Flow:

1. **Add Route:** Admin enters Origin/Destination → Stored in routes table.
2. **Create Flight:** Admin assigns a Date to a Route → Stored in flights table.
3. **Passenger Books:** Passenger selects a Flight → Click "Book".
4. **Booking Stored:** Application ID generated → Stored successfully in bookings table.

7. Project Folder Structure

Adapted layout for the system:

Plaintext

```

AirlineReservationSystem/
├── src/
│   ├── db/
│   │   └── DBConnection.java
│   ├── dao/
│   │   ├── RouteDAO.java
│   │   ├── FlightDAO.java
│   │   └── BookingDAO.java
│   ├── model/
│   │   ├── Route.java
│   │   ├── Flight.java
│   │   └── Booking.java
│   └── Main.java
├── lib/
│   └── mysql-connector-j-9.0.0.jar
└── database.sql

```

8. Design Rules Followed

- No SQL queries in UI layer.
- DAO layer strictly handles database.
- Use of primary and foreign keys.
- Clean separation of layers.